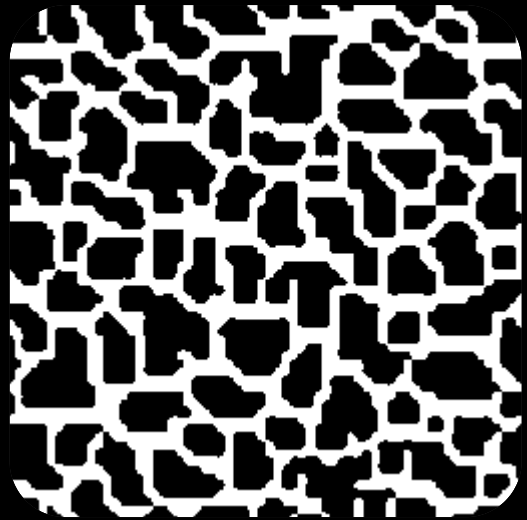




Study the attached reference image.

Develop a procedural Python generator that produces new binary microstructures similar to the reference. The outputs should be new stochastic realizations rather than copies.

Produce one complete, self-contained Python script.

The script must:

1. Generate twenty independent 256×256 binary microstructures.
2. Use random seeds 0 through 19.
3. Save each sample as both PNG and NPY.
4. Store solid as 1 and void as 0.
5. Generate genuinely different stochastic realizations.
6. Not read or access the reference image at runtime.
7. Not embed the reference image or a traced copy in the source code.
8. Not use internet access, external data, pretrained models, or APIs.
9. Use common Python scientific packages only.
10. Create all outputs from a procedural algorithm.
11. Include a short description of the algorithm and its adjustable parameters.
12. Return the complete script rather than fragments or pseudocode.

Do not ask questions. Select a reasonable procedural strategy based only on the image and implement it.

Show less

Worked for 4m 50s

generate_microstructures.py

Python

Run

```
#!/usr/bin/env python3
"""
Generate 20 independent 256 x 256 binary foam-like microstructures.

Algorithm
-----
1. Place a stochastic, jittered set of generator points on a staggered grid.
2. Warp the image coordinates with two smooth Gaussian random fields.
3. Compute a Voronoi-like cell labelling in the warped coordinates.
4. Detect cell interfaces and turn them into a connected solid network.
5. Vary the local interface thickness with coarse and fine random fields.

The result is an irregular cellular microstructure with a white, connected
solid phase and black void cells. The NumPy files store solid as 1 and void as
0 using uint8. PNG files use 255 for solid and 0 for void.

Main adjustable parameters
-----
CELL_SPACING:
    Typical centre-to-centre distance between void cells. Larger values make
    fewer, larger cells.
POINT_JITTER:
    Random displacement of generator points as a fraction of CELL_SPACING.
WARP_AMPLITUDE and WARP_SIGMA:
    Strength and correlation length of the smooth coordinate distortion.
SOLID_HALF_WIDTH:
    Base half-thickness, in pixels, of the solid interface network.
WIDTH_VARIATION and WIDTH_SIGMA:
    Magnitude and correlation length of slow thickness variation.
ROUGHNESS_AMPLITUDE and ROUGHNESS_SIGMA:
    Magnitude and scale of fine boundary roughness.

Dependencies: NumPy, SciPy, and Pillow.
"""

from __future__ import annotations

import argparse
import hashlib
```

```

from pathlib import Path

import numpy as np
from PIL import Image
from scipy import ndimage
from scipy.spatial import cKDTree

# Required output geometry and seeds.
IMAGE_SIZE = 256
SEEDS = range(20)

# Morphology parameters. These defaults are tuned for a dense, irregular,
# cellular network with approximately 90-100 void cells per image.
CELL_SPACING = 30.0
POINT_JITTER = 0.38
POINT_DROPOUT = 0.03
WARP_AMPLITUDE = 8.0
WARP_SIGMA = 30.0
SOLID_HALF_WIDTH = 2.8
WIDTH_VARIATION = 0.75
WIDTH_SIGMA = 18.0
ROUGHNESS_AMPLITUDE = 0.75
ROUGHNESS_SIGMA = 1.8
MIN_HALF_WIDTH = 1.6
MAX_HALF_WIDTH = 5.2

def _smooth_standard_normal(
    rng: np.random.Generator,
    shape: tuple[int, int],
    sigma: float,
) -> np.ndarray:
    """Return a zero-mean, unit-standard-deviation smooth random field."""
    field = rng.standard_normal(shape)
    field = ndimage.gaussian_filter(field, sigma=sigma, mode="reflect")
    field -= field.mean()

    standard_deviation = field.std()
    if standard_deviation < 1.0e-12:
        raise RuntimeError("Degenerate random field encountered.")

    field /= standard_deviation
    return field

def _make_generator_points(
    rng: np.random.Generator,
    size: int,
    spacing: float,
    jitter_fraction: float,
    dropout_probability: float,

```

```

) -> np.ndarray:
    """Create irregular staggered generator points beyond the image boundary."""
    margin = 1.5 * spacing

    x_positions = np.arange(
        -margin,
        size + margin + spacing,
        spacing,
    )
    y_positions = np.arange(
        -margin,
        size + margin + spacing,
        spacing,
    )

    points: list[tuple[float, float]] = []

    for row_index, y_base in enumerate(y_positions):
        stagger = 0.5 * spacing if row_index % 2 else 0.0

        for x_base in x_positions:
            # A small random dropout increases cell-size variability without
            # allowing the point process to become strongly clustered.
            if rng.random() < dropout_probability:
                continue

            x = (
                x_base
                + stagger
                + rng.uniform(-jitter_fraction, jitter_fraction) * spacing
            )
            y = (
                y_base
                + rng.uniform(-jitter_fraction, jitter_fraction) * spacing
            )

            points.append((x, y))

    if len(points) < 4:
        raise RuntimeError("Too few generator points were created.")

    return np.asarray(points, dtype=np.float64)

def generate_microstructure(seed: int) -> np.ndarray:
    """
    Generate one 256 x 256 microstructure as uint8 values in {0, 1}.

    Value 1 is solid; value 0 is void.
    """
    rng = np.random.default_rng(seed)
    shape = (IMAGE_SIZE, IMAGE_SIZE)

```

```

points = _make_generator_points(
    rng=rng,
    size=IMAGE_SIZE,
    spacing=CELL_SPACING,
    jitter_fraction=POINT_JITTER,
    dropout_probability=POINT_DROPOUT,
)

# Smooth random coordinate warping bends the otherwise straight Voronoi
# interfaces while retaining a cellular topology.
displacement_x = _smooth_standard_normal(
    rng,
    shape,
    WARP_SIGMA,
)
displacement_y = _smooth_standard_normal(
    rng,
    shape,
    WARP_SIGMA,
)

yy, xx = np.mgrid[0:IMAGE_SIZE, 0:IMAGE_SIZE]

warped_coordinates = np.column_stack(
    (
        (xx + WARP_AMPLITUDE * displacement_x).ravel(),
        (yy + WARP_AMPLITUDE * displacement_y).ravel(),
    )
)

tree = cKDTree(points)
_, nearest_label = tree.query(warped_coordinates, k=1)
labels = nearest_label.reshape(shape)

# Mark all four-neighbour interfaces between unlike Voronoi labels.
interface = np.zeros(shape, dtype=bool)

vertical_change = labels[1:, :] != labels[:-1, :]
horizontal_change = labels[:, 1:] != labels[:, :-1]

interface[1:, :] |= vertical_change
interface[:-1, :] |= vertical_change
interface[:, 1:] |= horizontal_change
interface[:, :-1] |= horizontal_change

# Distance from every pixel to the nearest cell interface.
distance_to_interface = ndimage.distance_transform_edt(~interface)

# Coarse noise changes strut thickness gradually; fine noise adds local
# boundary irregularity. Width is clipped to preserve a robust network.
coarse_width_noise = _smooth_standard_normal(

```

```

        rng,
        shape,
        WIDTH_SIGMA,
    )
    fine_roughness = _smooth_standard_normal(
        rng,
        shape,
        ROUGHNESS_SIGMA,
    )

    local_half_width = (
        SOLID_HALF_WIDTH
        + WIDTH_VARIATION * coarse_width_noise
        + ROUGHNESS_AMPLITUDE * fine_roughness
    )

    local_half_width = np.clip(
        local_half_width,
        MIN_HALF_WIDTH,
        MAX_HALF_WIDTH,
    )

    solid = distance_to_interface <= local_half_width
    microstructure = solid.astype(np.uint8)

    # Enforce the required output contract.
    if microstructure.shape != shape:
        raise RuntimeError(
            f"Unexpected output shape: {microstructure.shape}"
        )

    if not np.all(
        (microstructure == 0) | (microstructure == 1)
    ):
        raise RuntimeError("Output is not binary.")

    return microstructure


def save_microstructure(
    array: np.ndarray,
    output_dir: Path,
    seed: int,
) -> None:
    """Save one realization as both NPY and black/white PNG files."""
    stem = f"microstructure_seed_{seed:02d}"

    np.save(
        output_dir / f"{stem}.npy",
        array,
        allow_pickle=False,
    )

```

```

# NPY values remain 0 and 1. PNG values are scaled to 0 and 255 so that
# void appears black and solid appears white in ordinary image viewers.
png_array = (array * 255).astype(np.uint8)

Image.fromarray(png_array).save(
    output_dir / f"{stem}.png",
    optimize=False,
)

def main() -> None:
    parser = argparse.ArgumentParser(
        description=(
            "Generate 20 stochastic 256x256 binary microstructures."
        )
    )

    parser.add_argument(
        "--output-dir",
        type=Path,
        default=Path("generated_microstructures"),
        help=(
            "Directory for PNG and NPY outputs "
            "(default: generated_microstructures)."
        ),
    )

    args = parser.parse_args()
    args.output_dir.mkdir(parents=True, exist_ok=True)

    content_hashes: set[str] = set()

    for seed in SEEDS:
        microstructure = generate_microstructure(seed)

        # A duplicate is extraordinarily unlikely, but this check guarantees
        # that the delivered batch contains distinct realizations.
        digest = hashlib.sha256(
            microstructure.tobytes()
        ).hexdigest()

        if digest in content_hashes:
            raise RuntimeError(
                f"Duplicate realization detected at seed {seed}."
            )

        content_hashes.add(digest)

    save_microstructure(
        microstructure,
        args.output_dir,

```

```
        seed,
    )

    solid_fraction = float(microstructure.mean())

    print(
        f"seed {seed:02d}: saved PNG and NPY; "
        f"solid fraction = {solid_fraction:.4f}"
    )

    print(
        f"Created 40 files in: "
        f"{args.output_dir.resolve()}"
    )
```

```
if __name__ == "__main__":
```

ChatGPT can make mistakes. Check important info.